НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені Ігоря СІКОРСЬКОГО»
Навчально-науковий фізико-технічний інститут
Кафедра математичних методів захисту інформації

Звіт до лабораторної №4

за темою:
Дослідження систем захисту захищених месенджерів

Оформлення звіту:
Юрчук Олексій, ФІ-52мн

4 листопада 2026 р.
м. Київ

ЗМІСТ

1 Мета **1**

2 Принципи наскрізного шифрування (End-to-End Encryption) **1**

3 Порівняння безпечних систем обміну повідомленнями **3**

4 Аналіз ситуацій пов'язаних з порушенням безпеки **5**

4.1 Випадок 1: Компроментація сервера з повідомленнями 5

4.2 Ситуація 2: Злам кінцевого пристрою 6

4.3 Ситуація 3: Компрометація ключового матеріалу 6

4.4 Ситуація 4: Man-in-the-Middle Attack 7

4.5 Ситуація 5: Резервні копії повідомлень 8

5 Постквантовий захист систем обміну повідомленнями **9**

6 Висновки **10**

Список літературних і не дуже джерел **12**

1 Мета

Метою цієї лабораторної роботи є вивчення принципів, на яких побудовані системи безпечного обміну повідомленнями, механізмів наскрізного шифрування, моделей загроз для месенджерів та особливостей реалізації криптографічного захисту в сучасних месенджерах.

Поставлені задачі на лабораторну:

1. Розглянути основні поняття наскрізного шифрування (ключі ідентифікації та сеансу, узгодження ключів через untrusted server, Forward Secrecy, Post-Compromise Security, Double Ratchet, верифікація контактів) з використанням специфікацій протоколів;
2. Провести порівняння моделей безпеки Signal, WhatsApp, Telegram та Viber за вісьмома критеріями;
3. Проаналізувати п'ять ситуацій порушення безпеки, а також межі того, що може захистити E2EE;
4. Дослідити загрозу, яку квантові комп'ютери становлять для месенджерів («Harvest Now, Decrypt Later»), та заходи щодо її мінімізації в протоколі Signal.

2 Принципи наскрізного шифрування (End-to-End Encryption)

Я, традиційно для криптографії, використовував постаті Аліси і Боба, як двох користувачів.

Питання 1. Шифрування на рівні транспортного шару (TLS) проти наскрізного шифрування (E2EE). TLS (Transport Layer Security) захищає *канал* між двома кінцевими точками мережі, наприклад, телефоном Аліси та сервером месенджера: дані під час передачі є конфіденційними та захищеними від порушення цілісності, але на кінцевій точці сесія TLS завершується, і дані розшифровуються [1]. У месенджері цією кінцевою точкою є сервер, тому оператор бачить кожне повідомлення у вигляді відкритого тексту та зберігає його. E2EE (End-to-End Encryption) *переміщує кінцеві точки на пристрої користувачів*: повідомлення шифрується на пристрої Аліси за допомогою ключа, відомим лише пристроям Аліси та Боба, а сервер лише ретранслює зашифрований текст, який він не може прочитати [2]. Зазвичай обидва рівні використовуються разом: E2EE захищає вміст від сервера, а TLS (або «Noise» у WhatsApp) додатково приховує трафік від спостерігачів у мережі [3].

Приклад: Наглядно, можна навести приклад, що лише з TLS повідомлення схоже на лист у зачиненому броньованому фургоні, який відкривають на пошті; з E2EE сам лист знаходиться у сейфі, який може відкрити лише одержувач, а пошта бачить лише сейф.

Питання 2. Ключі ідентифікації та сесійні ключі. Ключ ідентифікації (IK) (**identity key (IK)**) – це довгострокова пара відкритого/закритого ключів (наприклад, Curve25519), що генерується на пристрої під час реєстрації. Його відкрита частина надсилається на сервер і криптографічно представляє користувача: вона використовується для автентифікації узгодження ключів та для підписання попередніх ключів користувача, і саме її користувачі порівнюють під час взаємної верифікації [2]. Сесійні ключі (СК) (**Session keys**) – це короткочасні *симетричні* ключі, що виводяться з угоди про ключі для конкретного діалогу; саме вони шифрують повідомлення і постійно замінюються – у протоколі Signal кожне повідомлення отримує власний ключ повідомлення [4]. Таке розділення означає, що ключ ідентифікації використовується рідко і лише для автентифікації, тоді як часто використовувані ключі шифрування легко змінювати та видаляти.

Формування спільного секрету через ненадійну мережу або сервер. Основою є **Diffie–Hellman (DH) key exchange**: кожна сторона має приватне значення і надсилає лише відповідне йому публічне значення; з «my private + your public» обидва обчислюють один і той самий секрет, тоді як спостерігач, який бачить лише публічні значення, цього зробити не може [2]. Месенджери повинні працювати й тоді, коли Боб перебуває в автономному режимі, тому вони використовують асинхронний варіант – X3DH (Extended Triple Diffie–Hellman) та його постквантовий наступник PQXDH. Боб завантажує на сервер

набір попередніх ключів: свій ідентифікаційний ключ, підписаний попередній ключ (signed prekey) та одноразові попередні (one-time prekeys) ключі (у PQXDH також постквантовий попередній ключ KEM). Аліса завантажує цей набір, перевіряє підпис, виконує кілька операцій за Діффі–Хелманом (між своїми ідентифікаційним та ефемерним ключами та ключами Боба), а в PQXDH додатково інкапсулює ще секрет у ключ KEM Боба; усі результати надходять у функцію KDF (Key Derivation Function), яка генерує спільний ключ SK [5]. Сервер зберігає та пересилає лише відкриті значення, тому він не може обчислити SK. Все, що він може зробити це замінити ключі Боба своїми власними, і це виявляється лише під час верифікації (спойлер до восьмого питання) [2].

Питання 4. Попередня таємність (Forward Secrecy). **Forward Secrecy (FS)** означає, що *компрометація поточних* ключів *не розкриває минулі повідомлення* [4]. Це досягається шляхом виведення нових ключів за допомогою односторонніх функцій та негайного видалення старих: з сьогоднішнього ключа обчислювально неможливо повернутися до вчорашнього. Для месенджерів це має вирішальне значення, оскільки розмови тривають роками, пристрої гублятся, а трафік може записуватися: без FS один викрадений ключ дозволив би розшифрувати всю записану історію.

Приклад: Якщо телефон Аліси було зламано в п'ятницю, повідомлення з понеділка по четвер, ключі до яких уже було видалено, залишаються нечитабельними для зловмисника, який їх зберіг.

Питання 5. Безпека після компрометації (Post-Compromise Security) та її відмінності від "Forward Secrecy" **Post-Compromise Security (PCS)**, яку також називають «break-in recovery» (відновлення після зламу) або «self-healing» (самовідновлення), означає, що після того, як зловмисник дізнався про поточний стан сеансу, безпека для майбутніх повідомлень *відновлюється*, коли сторони здійснюють обмін ключами з використанням нових випадкових даних, які зловмисник активно не спостерігав [6, 4]. FS захищає минуле від компрометації в сьогоденні; PCS захищає *майбутнє* після того, як компрометація завершилася. Система може мати FS без PCS: простий геш-ланцюг $k_{i+1} = H(k_i)$ забезпечує FS, але той, хто викраде $k(i)$, зможе обчислити усі майбутні ключі [4].

Питання 6. Чому ключі змінюються під час розмови? Зміна ключів обмежує «blast radius» (радіус ураження) будь-якого окремого ключа. Кожен ключ захищає лише кілька повідомлень, тому витік розкриває лише їх. Видалення використаних ключів забезпечує FS, а введення нових секретів через DH (або KEM) у ланцюг забезпечую PCS [4]. Навіть протоколи без full ratchet періодично змінюють ключі: у секретних чатах Telegram це оновлення ключів відбувається після 100 повідомлень або через тиждень [7], а ось групові дзвінки в Signal генерують новий медіа-ключ щоразу, коли хтось приєднується або виходить, щоб колишній учасник не міг далі підслуховувати розмову [8].

Питання 7. Проблема, яку вирішує Double Ratchet. The Double Ratchet поєднує два «two ratchets» – однонаправлені механізми (turn only forward) [4]: • «ратчет» із симетричним ключем генерує новий ключ повідомлення для кожного повідомлення з ланцюга KDF і видаляє його після використання — це забезпечує «Forward Secrecy» та дозволяє розшифровувати повідомлення, що надійшли не по порядку; • «ратчет» DH: кожна відповідь містить новий ефемерний відкритий ключ, а новий результат DH вбудовується в кореневий ланцюг — це вносить свіжу випадковість і забезпечує безпеку після компрометації.

- **Symmetric-key ratchet** генерує новий ключ повідомлення для кожного повідомлення з ланцюга KDF і видаляє його після використання – це забезпечує «Forward Secrecy» та дозволяє розшифровувати повідомлення, що надійшли не по порядку;
- **DH ratchet**: кожна відповідь містить новий ефемерний відкритий ключ, а новий результат DH вбудовується в кореневий ланцюг – це вносить fresh randomness і забезпечує безпеку після компрометації (Post-Compromise Security).

Проблема, яку це вирішує, це *long-lived asynchronous conversations* тривалі асинхронні діалоги: користувачі не перебувають у мережі одночасно, повідомлення надходять із запізненням або у неправильному порядку, але при цьому кожне повідомлення має мати власний ключ, старі ключі мають зникати, а вкрадені дані мають втрачати корисність, щойно діалог продовжується [4]. Таку саму конструкцію використовують WhatsApp та Viber (у власній реалізації) [3, 9].

Питання 8. Верифікація контакту: код безпеки, відбиток пальця, QR-код. Сервер розповсюджує відкриті ключі ідентифікації, тому зловмисний або скомпрометований сервер міг би передати Алісі свій власний ключ замість ключа Боба та втрутитися в процес (створивши MITM, man-in-the-middle). Шифрування все одно ”працюватиме”, але з неправильним учасником (В общєм погано). Верифікація усуває цю вразливість: обидва пристрої обчислюють коротке значення на основі двох ідентифікаційних ключів – у Signal це 60-значний код (safety number) або QR-код, у WhatsApp – 60-значний код безпеки, у Telegram – чотири довільні емодзі, у Viber – 48-значний номер, який порівнюється під час дзвінка [10, 3, 11, 9]. Якщо значення збігаються під час порівняння через незалежний канал (наприклад, обговорили усно), це означає, що ніхто не підмінив ключі [2]. Оскільки ручне порівняння проводиться рідко, WhatsApp і Signal додали прозорість ключів (key transparency): публічний каталог ключів, до якого можна лише додавати записи, який клієнти перевіряють автоматично, щоб сервер не міг непомітно показувати різні ключі різним користувачам [12, 13].

3 Порівняння безпечних систем обміну повідомленнями

У таблиці 1 я порівнював моделі безпеки Signal, WhatsApp, Telegram та Viber. Наведено лише ті властивості, які задокументовані розробниками або незалежними організаціями; якщо документація не містить інформації, про це зазначено окремо.

Таблиця 1: Порівняння моделей безпеки Signal, WhatsApp, Telegram та Viber.

Критерій	**Signal**	**WhatsApp**	**Telegram**	**Viber**
E2EE у приватних чатах	Так, протокол Signal: PQXDH + Double Ratchet + SPQR [5, 14]	Так, протокол Signal (X3DH + Double Ratchet) [3]	Тільки в *секретних чатах* (MTProto 2.0, DH + AES-256-IGE) [7]	Так, власна реалізація алгоритмів Double Ratchet, Curve25519, Salsa20 [9]
E2EE за замовчуванням	Так, для всіх чатів та дзвінків	Так, для особистих та групових чатів	Ні: хмарні чати використовують шифрування «клієнт–сервер»; секретні чати є опціональними та прив’язаними до пристрою [15]	Так, для індивідуальних та групових чатів і індивідуальних дзвінків [16]

Continued on next page

Таблиця 1: Порівняння моделей безпеки Signal, WhatsApp, Telegram та Viber. (Continued)

Критерій	Signal	WhatsApp	Telegram	Viber
Групові чати	E2EE; склад учасників зберігається у зашифрованому вигляді, сервер не знає учасників групи [17]	E2EE через *відправника*; сервер розшифровує одне зашифроване повідомлення, тому знає склад учасників; ключі скидаються, коли учасник залишає групу [3]	Без E2EE, лише клієнт–сервер [15]	E2EE через спільний секрет групи, що розподіляється по парних сесіях [9]
Голосові / відеодзвінки	E2EE, у тому числі групові дзвінки (шифрування кадрів, SFU не може розшифровувати, оновлення ключа при приєднан-ні/виході) [8]	E2EE, SRTP із випадковим головним секретом, надісланим у зашифрованому повідомленні [3]	Дзвінки 1:1 — E2EE; групові дзвінки E2EE до 200 осіб з 2025 року [11, 18]	Дзвінки 1:1 — E2EE: SRTP + Salsa20 із підписаними тимчасовими ключами [9]
Перевірка контактів	60-значний код безпеки / QR-код; автоматична перевірка ключів (прозорість ключів) [10, 13]	60-значний код / QR-код; прозорість ключів (Auditable Key Directory) [3, 12]	Зображення ключа (288-бітний хеш) у секретних чатах; 4 емодзі під час дзвінків [7, 11]	48-значний номер, що порівнюється під час дзвінка; сірий / зелений / червоний замок [9]
Попередня секретність	Так, для кожного повідомлення; також постквантова FS та PCS [14]	Так у чатах 1:1 (Double Ratchet); у групах — FS у межах ланцюга відправників, PCS — лише після скидання ключа [3]	Тільки для секретних чатів: оновлення ключа кожні 100 повідомлень / 1 тиждень; відсутнє для хмарних чатів [7]	Так, заявлена пряма та зворотна секретність [9]
Історія та резервні копії	Історія лише на пристроях; опціональні безпечні резервні копії, зашифровані за допомогою 64-символьного ключа відновлення, який ніколи не залишає пристрою [19]	Резервні копії в iCloud / Google Drive; резервне копіювання з E2EE є опціональним (64-значний ключ або пароль у сховищі HSM) [20, 21]	Хмарні чати зберігаються на серверах Telegram (ключі розподілені між юрисдикціями); секретні чати не зберігаються у хмарі [15]	Резервні копії в Google Drive / iCloud; відсутність задокументовано-го E2EE — доступ до хмарного облікового запису надає доступ до історії [22, 23]

Continued on next page

Таблиця 1: Порівняння моделей безпеки Signal, WhatsApp, Telegram та Viber. (Continued)

Критерій	Signal	WhatsApp	Telegram	Viber
Дані доступні оператору, незважаючи на E2EE	Адресат, IP-адреса та час; зашифрований відправник приховує особу відправника [24]	Номери телефонів, хто з ким і коли спілкується, членство в групах, IP-адреси, відкриті ключі [3]	Для «хмарних чатів» — *вміст* та всі метадані; для «секретних чатів» — метадані	Номери телефонів, контакти, час; IP-адреса, видима партнеру під час P2P-дзвінків [23]

Висновки.. За словом ”зашифрований” ховаються у кожного дуже різні моделі безпеки. Signal за замовчуванням застосовує E2EE до всього, мінімізує метадані та є відкритим програмним забезпеченням, тому його нюанси реалізації можна перевірити власноруч; WhatsApp використовує той самий протокол для вмісту, але залишає оператору розгалужену соціальну мережу та зберігає резервні копії поза межами E2EE, якщо користувач не увімкне цю функцію; Telegram шифрує більшість чатів лише між клієнтом та власними серверами, тому за звичайних умов використання оператор (та будь-хто, хто змушує його до цього або отримує доступ) може вільно читати вміст, тоді як E2EE обмежується опціональними секретними чатами 1 на 1 та дзвінками; Viber має E2EE за замовчуванням, але з закритою реалізацією та хмарними резервними копіями, які не задокументовані як end-to-end encrypted.

Отже, реальний рівень захисту визначається не наявністю шифру, а тим, *де знаходяться ключі*, *що ввімкнено за замовчуванням*, *що відбувається з резервними копіями та шифрування в групах*, а також *скільки метаданих бачить сервер* – тобто моделлю довіри, а не алгоритмом.

4 Аналіз ситуацій пов’язаних з порушенням безпеки

4.1 Випадок 1: Компроментація сервера з повідомленнями

Зловмисник контролює серверну інфраструктуру месенджера з правильною реалізацією E2EE (рис. 1).

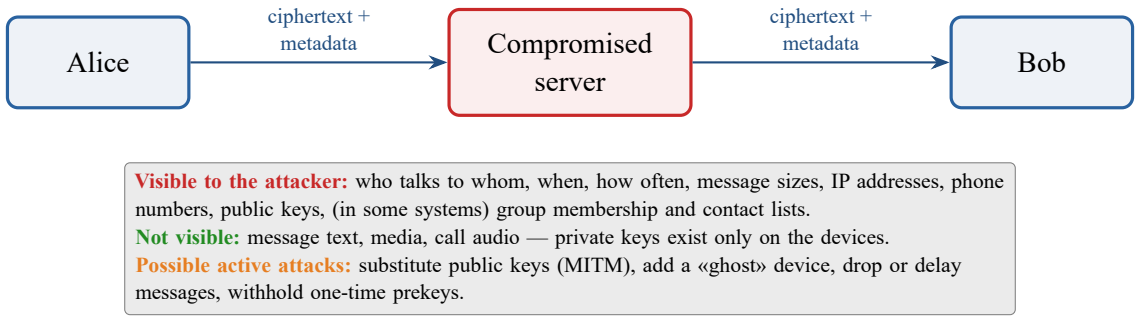


Рис. 1: Situation 1: what a compromised server can and cannot do under correct E2EE.

Чи може зловмисник прочитати вміст? Якщо коротко – ні. Приватні ідентифікаційні та сесійні ключі ніколи не залишають своїх клієнтів; сервер зберігає лише відкриті ключі та ретранслює зашифровані тексти [3, 2]. Навіть повна копія бази даних сервера не містить жодного ключа, який би розшифровував повідомлення.

Що може отримати зловмисник? Метадані: ідентифікатори (номери телефонів), соціальний граф (взаємозв’язки між людьми), час і розмір повідомлень, IP-адреси, інформація про пристрої та, залежно від архітектури, приналежність до груп. Signal обмежує це за допомогою функції ”sealed sender”(сервер бачить одержувача, але не відправника) та зашифрованої приналежності до груп [24, 17]; у WhatsApp сервер розсилає групові повідомлення, а отже, знає учасників груп [3].

Властивості E2EE, що обмежують збитки. Ключі генеруються та зберігаються лише на кінцевих пристроях; кожне повідомлення проходить автентифікацію, тому сервер не може непомітно підробляти чи змінювати повідомлення; принцип ”Forward Secrecy” означає, що записані шифровані тексти залишаються в безпеці, навіть якщо ключі згодом будуть скомпроментовані [4].

Чого не можна запобігти лише за допомогою E2EE. Активний сервер може замінити ідентифікаційний ключ Боба своїм власним і здійснити атаку ”людина посередині (MITM) під час першого контакту, яку можна виявити лише за допомогою верифікації або прозорості ключів (як я пізніше побачив у ситуації 4); він може додати прихований пристрій до облікового запису, якщо пристрої не верифікуються; він може відмовитися видавати одноразові попередні ключі, що послаблює FS для першого повідомлення [2]; а також може видаляти, затримувати або блокувати повторне відтворення повідомлень. E2EE також не може запобігти збиранню метаданих або розповсюдженню шкідливого оновлення клієнтського – для цього потрібні відкритий вихідний код та відтворювані збірки.

4.2 Ситуація 2: Злам кінцевого пристрою

Зловмисник має доступ до розблокованого смартфона користувача(рис. 2). Ну, тут наче все очевидно буде:

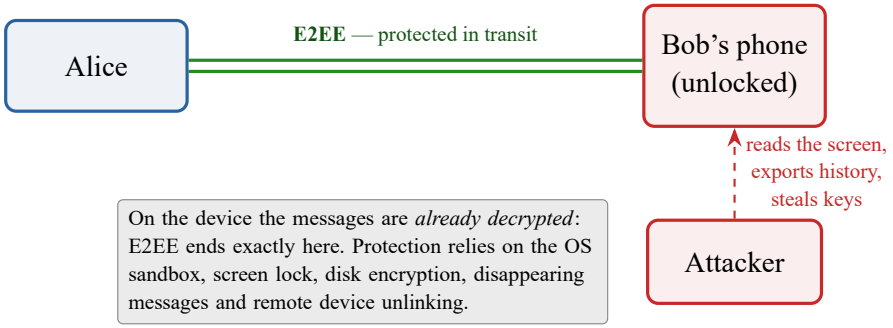


Рис. 2: Situation 2: the end device is the boundary of end-to-end encryption.

Чи захищає E2EE? Ні. E2EE захищає дані *між* кінцевими точками; розблокований телефон є сам по собі кінцевою точкою, де історія зберігається у розшифрованому вигляді, а приватні ключі доступні для додатка. Зловмисник може прочитати всі збережені повідомлення, зробити знімки екрана, надсилати повідомлення від імені користувача, підключити новий пристрій до облікового запису та скопіювати ключові матеріали. В общем, майже все що захоче.

У специфікації Double Ratchet чітко зазначено, що компрометація ключів або цілісності пристрою має серйозні наслідки, і сторона, що зазнала компрометації, повинна негайно(!) замінити свої ключі [4].

А що ж все ж захищено..? ”Forward Secrecy” захищає лише ті повідомлення, які вже були видалені з пристрою (наприклад, disappearing messages), оскільки їхні ключі та відкритий текст вже відсутні; PCS відновлює сеанс після того, як зловмисник втрачає доступ, але лише за умови, що він не встановив стійке шкідливе програмне забезпечення [4]. Тому безпека пристрою є окремим компонентом: блокування екрана та повне шифрування диска, блокування на рівні додатків, повідомлення, що зникають, перелік підключених пристроїв із можливістю їх від’єднання, а також оновлення ОС для захисту від шпигунського програмного забезпечення. Криптографія ну ніяк не може захистити пристрій, який контролюється повністю зловмисником.

4.3 Ситуація 3: Компрометація ключового матеріалу

Зловмисник отримав ключовий матеріал одного сеансу (наприклад, ланцюговий ключ у момент часу t). На рис. 3 показано наслідки залежно від властивостей протоколу, що використовується.

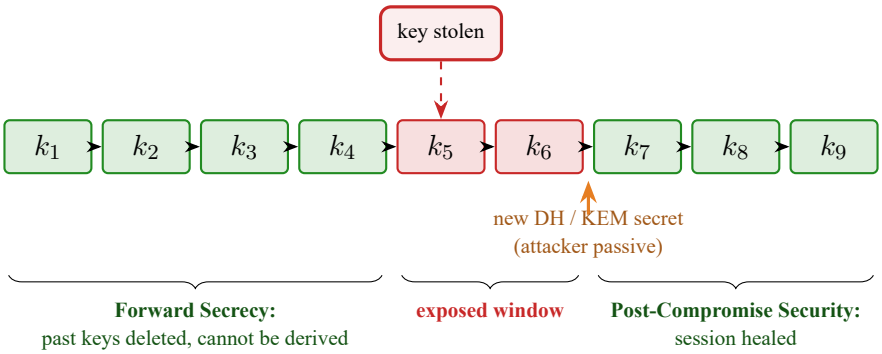


Рис. 3: Situation 3: consequences of a key compromise in a ratcheting protocol.

Коли зловмисник може розшифрувати попередні повідомлення. Виключно за умови, якщо протокол не має функції Forward Secrecy (наприклад, один і той самий довгостроковий ключ шифрує всю історію), або якщо старі ключі повідомлень чи відкриті тексти не були видалені з пристрою. Наприклад, ключі пропущених повідомлень зберігалися занадто довго або наявна незашифрована локальна історія [4]. У разі правильного Double Ratchet ключі до деякого моменту часу t були отримані за допомогою односторонніх функцій і видалені, тому їх неможливо відновити.

Коли зловмисник може розшифрувати усі наступні повідомлення. У випадку суто симетричного ланцюга викрадений ключ ланцюга дозволяє обчислити всі майбутні ключі повідомлень [4]. У алгоритмі Double Ratchet це можливо лише до наступного DH ratchet кроку: щойно сторона надсилає повідомлення з новим ефемерним ключем, у кореневий ланцюг потрапляє новий секрет DH, невідомий зловмиснику, і сесія захищено відновлюється. Зловмисник зберігає доступ лише в тому випадку, якщо може діяти *активно*. Наприклад, продовжуючи видавати себе за одну зі сторін або викравши довгостроковий ідентифікаційний ключ і здійснюючи атаку "людина посередині" на нові сесії [6].

Роль механізмів. Forward Secrecy обмежує збитки, завдані в минулому; часта зміна ключів обмежує використання кожного ключа кількома повідомленнями; Post-Compromise Security (захист після компрометації) обмежує збитки, які можуть потенційно бути завдані в майбутньому. Разом вони скорочують вікно вразливостей до періоду між two ratchet steps. Слабші архітектури мають ширше вікно: у секретних чатах Telegram перегенерація ключів відбувається, як я вже згадував вище, лише після 100 повідомлень або через тиждень [7]; Sender Keys у WhatsApp не забезпечують відновлення, доки ключ відправника не буде скинуто [3].

4.4 Ситуація 4: Man-in-the-Middle Attack

Аліса та Боб вперше встановлюють захищене з'єднання через сервер, якому вони не довіряють повністю (рис. 4).

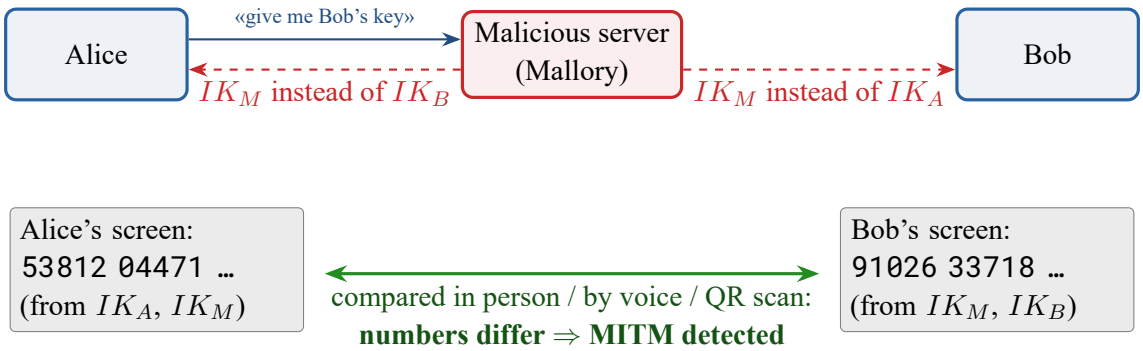


Рис. 4: Situation 4: key substitution by the server and its detection by comparing safety numbers.

Чому зашифроване з'єднання не підтверджує ідентичність? Узгодження ключів гарантує, що Аліса має *спільний секрет із власником отриманого нею відкритого ключа*. Але ключ надійшов від сервера. Якщо сервер повернув свій власний ключ IK_M, Аліса здійснює цілком коректне узгодження ключів із Маллорі, Маллорі робить те саме з Бобом і повторно шифрує повідомлення між ними. Обидва бачать «end-to-end encrypted», проте зловмисник читає все. У специфікаціях X3DH та PQXDH зазначено, що без порівняння ключів ідентифікації через аутентифікований (або інший сторонній) канал сторони *не отримують жодної криптографічної гарантії* щодо того, з ким саме вони спілкуються [2, 5]. Більшість месенджерів використовують TOFU (Trust On First Use): перший ключ приймається, а про подальші зміни повідомляється лише згодом.

Як система верифікації виявляє атаку. Код безпеки, відбиток або QR-код обчислюються на основі *обох ключів* ідентифікації, які пристрій фактично використовує. Наприклад, у WhatsApp шляхом гешування кожного ключа ідентифікації з ідентифікатором користувача 5 200 разів за допомогою алгоритму SHA-512 та об'єднання двох 30-значних частин [3]. У разі атаки "людина посередині" (MITM) значення Аліси обчислюється з (IK_A, IK_M), а Боба – з (IK_M, IK_B), тому вони не можуть збігатися. Порівняння має відбуватися через канал, який сервер не контролює: особисто (наприклад, сканування QR-коду), голосом під час дзвінка (Viber, 4 емодзі Telegram) або через інший надійний канал [10, 9, 11]. Після перевірки будь-яка подальша зміна ключа відображається як попередження (Signal надсилає сповідення "номер безпеки змінився", Viber – червоний замочок). Прозорість ключів (Key transparency) автоматизує цей процес для користувачів, які ніколи не порівнюють номери: сервер повинен оприлюднювати кожен ключ у каталозі, до якого можна *лише додавати* записи та який підлягає публічному аудиту, тому якщо він показуватиме різні ключі різним користувачам, це залишатиме докази присутності зловмисника [12, 13].

4.5 Ситуація 5: Резервні копії повідомлень

Месенджер використовує E2EE для передачі даних, але історія користувача зберігається у резервній копії (рис. 5).

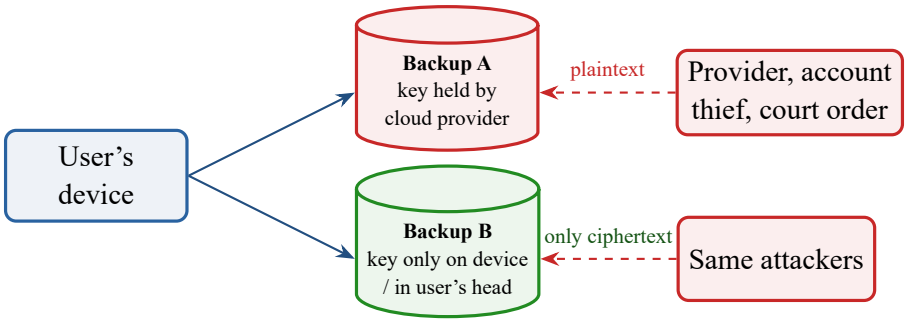


Рис. 5: Situation 5: a backup protected by the provider (A) vs. an end-to-end encrypted backup (B).

Чому E2EE під час передачі не захищає резервну копію. E2EE закінчується на пристрої, де повідомлення розшифровуються. Резервна копія – це *нова копія* цих відкритих текстів, надіслана до сторонньої системи, наприклад, iCloud, Google Drive або сервера месенджера. Якщо її зашифровано ключем, який зберігається в цій системі, захист залежить від провайдера та від безпеки хмарного облікового запису користувача, а не від E2EE. WhatsApp стверджує, що його стандартні резервні копії захищені окремими хмарними сервісами зберігання даних [20]; у Viber організація Privacy International попереджає, що будь-хто, хто отримає доступ до облікового запису Apple або Google, зможе отримати доступ до всіх розмов (оскільки увімкнене резервне копіювання саме туди) [23]. У Telegram Cloud Chats окремої резервної копії взагалі немає – історія постійно зберігається на сервері [15]. Таким чином, резервне копіювання може непомітно звести нанівець усі переваги E2EE, включаючи Forward Secrecy: оскільки повідомлення, давно видалені з чату, залишаються доступними для читання в архіві.

Вимоги до резервної копії з наскрізним шифруванням (end-to-end protected backup).

- Резервна копія *шифрується на пристрої* за допомогою надійного випадкового ключа перед тим, як покинути його;
- Ключ ніколи не є доступним для провайдера: його або зберігає користувач (64-символьний ключ відновлення у Signal; 64-цифровий ключ у WhatsApp) [19, 20], або він захищений паролем користувача всередині апаратних модулів безпеки (Hardware Security Modules, HSM), які оператор не може прочитати і які обмежують кількість спроб підбору, а потім знищують ключ [20, 21];
- Шифрування з автентифікацією, завдяки чому резервну копію неможливо змінити непомітно; заповнення для приховування розмірів (Signal заповнює медіафайли) [19];
- можливість перевірки інфраструктури HSM (Meta публікує докази кожного нового розгортання парку HSM) [21];
- чесний компроміс: якщо ключ втрачено, резервна копія втрачається назавжди [19].

Крім того, *надійність* резервної копії *визначається найслабшим ланцюгом*: якщо Аліса використовує резервні копії з кінцевим шифруванням (E2EE), а Боб – звичайну хмарну резервну копію з шифруванням на стороні сервера, їхня розмова все одно залишається вразливою через Боба.

5 Постквантовий захист систем обміну повідомленнями

Сценарій "Зібрати зараз, розшифрувати пізніше" (The Harvest Now, Decrypt Later, HNDL). Цей принцип означає, що зловмисник сьогодні записує зашифрований трафік і зберігає його доти, доки не з'явиться cryptographically relevant quantum computer(CRQC); після цього він розшифровує обмін ключами та розшифровує архів. NIST наголошує, що через цю загрозу – сьогоднішні зашифровані дані вже знаходяться під загрозою [25]. Для месенджерів це критично: розвідувальне агентство, що контролює магістральну мережу, може непомітно збирати всі шифровані тексти Аліси та Боба, а також значення відкритих ключів, що використовуються для обміну.

2. Чому загроза є реальною ще до появи квантового комп'ютера? Важливим є не момент появи CRQC, а те, як довго дані мають залишатися таємними. Якщо приватне листування має залишатися конфіденційним протягом 15–20 років, а CRQC з'явиться через 10 років, дані, надіслані сьогодні, вже є вразливими. Збір цих даних відбувається пасивно й непомітно, а міграція великих систем займає роки. NIST IR 8547, остаточна редакція документу була затверджена буквально декілька днів тому, описує цей перехід, причому міграція федеральних систем обміну ключами США для систем високої важливості має бути завершена до 2030 року [25]. Ось чому захист обміну ключами необхідно впровадити вже зараз.

3. Механізми, що потребують постквантового захисту. Алгоритм Шора розв'язує задачі, засновані на дискретних логарифмах та розкладі числа на множники, тобто по факту всю криптографію на еліптичних кривих, що використовується в месенджерах:

- **узгодження ключів** – X25519 в X3DH та на кожному етапі DH алгоритму Double Ratchet; це найнагальніша частина, оскільки вона вразлива до HNDL [5];
- **підписи та автентифікація** – ключі ідентифікації, signed prekeys, сертифікати; їм загрожує лише активний зловмисник, який має квантовий комп'ютер під час зв'язку, тому вони є менш нагальними щодо HNDL [5];
- **transport** (TLS, Noise) та обмін ключами для дзвінків, резервного копіювання та зв'язку між кількома пристроями.

Симетричні примітиви (AES-256, SHA-256, HMAC) ослаблюються лише алгоритмом Гровера і залишаються досі безпечними при використанні 256-бітних ключів.

4. Інтеграція в Signal. Signal додає постквантову криптографію у два етапи, завжди у emphhybrid режимі (рис. 6):

- **PQXDH** (2023) захищає початкову домовленість щодо ключа. Боб додатково публікує підписаний преключ KEM (Kyber / ML-KEM). Аліса інкапсулює в нього випадковий секретний SS, і сесійний

ключ дорівнює $SK = \text{KDF}(DH_1 \| DH_2 \| DH_3 \| DH_4 \| SS)$ [5]. ML-KEM – це механізм інкапсуляції ключів, стандартизований NIST у FIPS 203 [26].

- SPQR (Sparse Post-Quantum Ratchet, жовтень 2025) захищає решту розмови. Він працює паралельно з "Double Ratchet", періодично встановлюючи нові секрети ML-KEM-768; для кожного повідомлення ключі обох механізмів змішуються в KDF – *Triple Ratchet*. Оскільки ключі та шифровані тексти ML-KEM мають розмір близько 1 КБ замість 32 байтів, їх розділяють на фрагменти та надсилають разом зі звичайними повідомленнями з використанням кодів виправлення [14].

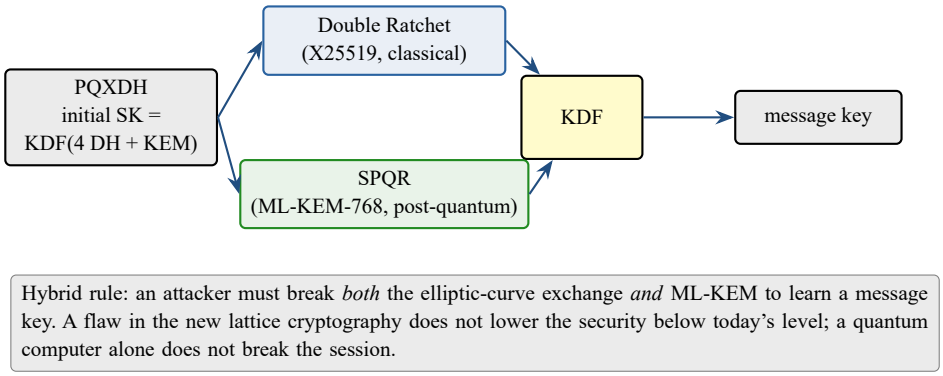


Рис. 6: Post-quantum integration in Signal: PQXDH for session setup and the Triple Ratchet for the conversation.

5. Додаткова властивість безпеки. Традиційні алгоритми забезпечують FS та PCS лише проти класичного зловмисника. Модернізація додає **постквантову конфіденційність**: записаний трафік не може бути розшифрований майбутнім квантовим комп'ютером (стійкість до HNDL), а завдяки SPQR також забезпечується **постквантова Forward Secrecy** та **постквантова Post-Compromise Security** – після компрометації сесія відновлюється за допомогою секретів, які квантовий зловмисник не може вивести з спостережуваного трафіку [5, 14].

Гібридна конструкція зберігає існуючі класичні гарантії, тому рівень безпеки ніколи не погіршується порівняно з попереднім станом. Залишається одне обмеження: автентифікація все ще є класичною. У PQXDH прямо зазначено, що активний зловмисник із квантовим комп'ютером може видати себе за одну зі сторін, а питання про постквантову взаємну автентифікацію з можливістю заперечення залишається відкритою проблемою [5].

6 Висновки

1. TLS захищає повідомлення лише до сервера; E2EE зміщує межу довіри на пристрої користувачів, тому сервер лише ретранслює зашифровані тексти. Ключі ідентифікації автентифікують користувачів, тоді як короткочасні сесійні ключі, отримані за допомогою асинхронного алгоритму Діффі–Хеллмана (X3DH/PQXDH) та оновлювані за допомогою алгоритму Double Ratchet, шифрують повідомлення та забезпечують пряму секретність і безпеку після компрометації.

2. E2EE завжди покладається на правильність відкритих ключів, наданих сервером. Коди безпеки, QR-коди та прозорість ключів. Це єдиний спосіб виявити атаку типу "людина посередині"; без перевірки зашифрований чат може бути зашифрований на користь зловмисника.

3. Наведені месенджери принципово відрізняються. Signal забезпечує E2EE за замовчуванням із мінімальним обсягом метаданих та постквантовим захистом; WhatsApp використовує той самий протокол, але залишає оператору значний обсяг метаданих і робить шифрування резервних копій необов'язковим; Telegram використовує E2EE лише в необов'язкових секретних чатах та дзвінках 1:1, тому більша частина його вмісту доступна оператору; Viber має E2EE за замовчуванням, але

закриту реалізацію та незахищені хмарні резервні копії. Отже, наявність шифрування не означає однакового рівня чи моделі захисту.

4. Аналіз випадків порушення безпеки показує обмеження E2EE: воно витримує компрометацію сервера (але не збір метаданих або активну заміну ключів), не захищає скомпрометований кінцевий пристрій, обмежує компрометацію ключів лише коротким проміжком часу завдяки ratcheting, вимагає перевірки на наявність атак типу ”людина посередині” (MITM) і втрачає ефективність через резервні копії, які не шифруються ключем, що зберігається виключно у користувача.

5. Загроза Harvest Now, Decrypt Later робить обмін постквантовими ключами необхідним ВЖЕ сьогодні. Гібридна технологія PQXDH та Triple Ratchet від Signal (SPQR з ML-KEM) додають потрібну постквантову конфіденційність, пряму секретність та безпеку після компрометації, зберігаючи при цьому класичні гарантії; Хоча постквантова автентифікація залишається невирішеним завданням.

Список літературних і не дуже джерел

[1] Eric Rescorla. *RFC 9846 — The Transport Layer Security (TLS) Protocol Version 1.3*. 2026. (URL) (visited on 11/04/2026).

[2] Moxie Marlinspike and Trevor Perrin. *The X3DH Key Agreement Protocol (Revision 1)*. Signal. 2016. (URL) (visited on 11/01/2026).

[3] WhatsApp. *WhatsApp Encryption Overview: Technical White Paper (Version 3)*. Archived copy; original at whatsapp.com/security. 2020. (URL) (visited on 11/03/2026).

[4] Trevor Perrin and Moxie Marlinspike. *The Double Ratchet Algorithm*. Signal. 2016. (URL) (visited on 11/01/2026).

[5] Ehren Kret and Rolfe Schmidt. *The PQXDH Key Agreement Protocol (Revision 3)*. Signal. 2024. (URL) (visited on 11/01/2026).

[6] Katriel Cohn-Gordon, Cas Cremers, and Luke Garratt. «On Post-Compromise Security». In: *2016 IEEE 29th Computer Security Foundations Symposium (CSF)*. IEEE, 2016. (URL) (visited on 11/04/2026).

[7] Telegram. *End-to-End Encryption, Secret Chats*. 2026. (URL) (visited on 11/04/2026).

[8] Signal. *How to Build Large-Scale End-to-End Encrypted Group Video Calls*. 2021. (URL) (visited on 11/01/2026).

[9] Rakuten Viber. *Viber Encryption Overview*. 2016. (URL) (visited on 11/01/2026).

[10] Signal Support. *What Is a Safety Number?* 2026. (URL) (visited on 11/01/2026).

[11] Telegram. *FAQ for the Technically Inclined*. 2026. (URL) (visited on 11/04/2026).

[12] Meta Engineering. *Deploying Key Transparency at WhatsApp*. 2023. (URL) (visited on 11/03/2026).

[13] Signal Support. *What Is Automatic Key Verification?* 2026. (URL) (visited on 11/01/2026).

[14] Graeme Connell and Rolfe Schmidt. *Signal Protocol and Post-Quantum Ratchets*. Signal. 2025. (URL) (visited on 11/01/2026).

[15] Telegram. *Telegram FAQ*. 2026. (URL) (visited on 11/04/2026).

[16] Rakuten Viber. *Private and Secure Messaging*. 2026. (URL) (visited on 11/01/2026).

[17] Signal. *Technology Preview: Signal Private Group System*. 2019. (URL) (visited on 11/01/2026).

[18] Telegram. *Extra-Secure Group Calls, Automated Accounts, and More*. 2025. (URL) (visited on 11/04/2026).

[19] Signal. *Introducing Signal Secure Backups*. 2025. (URL) (visited on 11/01/2026).

[20] Meta Engineering. *WhatsApp End-to-End Encrypted Backups*. 2021. (URL) (visited on 11/04/2026).

[21] Meta Engineering. *Strengthening End-to-End Encrypted Backups: The HSM-Based Backup Key Vault*. 2026. (URL) (visited on 11/04/2026).

[22] Rakuten Viber Support. *Back Up and Restore Your Chat History*. 2026. (URL) (visited on 11/01/2026).

[23] Privacy International. *Guide on Viber Settings and Good Practices*. 2024. (URL) (visited on 11/01/2026).

[24] Signal. *Technology Preview: Sealed Sender for Signal*. 2018. (URL) (visited on 11/01/2026).

[25] National Institute of Standards and Technology. *NIST Report Provides Guidelines for Transition to Post-Quantum Cryptography Standards (NIST IR 8547)*. 2026. (URL) (visited on 11/04/2026).

[26] National Institute of Standards and Technology. *FIPS 203 — Module-Lattice-Based Key-Encapsulation Mechanism Standard*. 2024. (URL) (visited on 11/04/2026).